#. We start by initializing the list of points found within this subtree.
#5 If, for each dimension `i`, the `i`-th coordinate of current node's point is within the rectangle's boundaries for that coordinate, then we can add current point to the results.
#6 If the rectangle boundaries intersect left branch, either being on the left of current node's split line in, or intersecting it, then we need to traverse left branch and add all points found to current results. We won't provide this helper function, for the sake of space, but you can easily write it using figure 9.26 as a reference.
#7 Similarly, if the rectangular search region intersects right branch...
#8 Finally, we return all points found in this subtree.

Therefore, we can only say that the worst-case running time is `O(n)`, even if the methods will efficiently prune the tree whenever possible.

9.3.7 A Recap of all Methods

As we have seen, k-d trees provides a speedup over brute-force search on the whole dataset: while the worst-case running time for nearest neighbor search (and removal) is still linear (exactly as for brute-force), in practice the amortized performance on balanced k-d trees is slightly-to-consistently better. The improvement is higher in low dimensional spaces, and still consistent in medium-dimensional spaces.

In high-dimensional spaces, the exponential term on `k` for nearest neighbor becomes dominant, and makes not-worthy